Universidad Diego Portales

Análisis empírico y teórico de algoritmos

Problema: Diferencias Repetidas

Estructuras de Datos y Algoritmos
2026-01

Integrantes:

Facundo Ahumada, Renato Saltzer, Lucas Yañez.

Índice

Introducción **2**

1. Etapa 1: Análisis empírico **3**
1.1. Estructura del software 3
1.2. Desafíos enfrentados 3
1.3. Análisis empírico 4

2. Etapa 2: Resultados en excel **6**

3. Etapa 3: Análisis teórico **7**
3.1. Método 1: Función cuentaPares 7
3.2. Método 2: Función cuentaPares2 8
3.3. Método 3: Función cuentaPares3 9

4. Etapa 4: Comparaciones **10**

5. Conclusiones **11**

Introducción

El presente trabajo aborda el problema computacional de las «Diferencias repetidas». Dada una colección C de n números enteros distintos y un valor k, se debe determinar la cantidad de pares cuya diferencia es igual a k. Para ello, se evaluarán tres métodos en Java. Cada uno presenta una lógica distinta y será analizado estrictamente sin modificar su código fuente original.

El objetivo principal de este laboratorio es realizar un análisis exhaustivo de la complejidad temporal de los tres algoritmos. Esto se logrará aplicando el ciclo metódico de estudio algorítmico: la medición empírica de los tiempos de ejecución, el análisis teórico del orden de crecimiento mediante la notación Big-O, y una contrastación final para verificar si la teoría matemática predice con exactitud el comportamiento observado en los experimentos.

Para lograr los objetivos señalados, se construyen y entregan los siguientes elementos:

- Un programa en Java encargado de generar instancias aleatorias y medir los tiempos de ejecución de cada método.
- Archivos en formato CSV que almacenan los tiempos registrados para cada tamaño de entrada n.
- Un archivo Excel que procesa los datos obtenidos mediante estadísticas descriptivas y visualizaciones gráficas.
- El presente informe en formato PDF, que documenta todo el proceso experimental y consolida los resultados y conclusiones obtenidas.

1. Etapa 1: Análisis empírico

1.1. Estructura del software

El programa utiliza cinco clases. La clase `Analisis` es el núcleo del experimento y contiene dos métodos: uno que genera los valores aleatorios sin repetición, y el método `main` que orquesta todo el experimento. Para evitar valores repetidos se utiliza un `HashSet`, que descarta automáticamente duplicados hasta completar n valores distintos.

Las instancias se generan dentro de un bucle que recorre cada tamaño n, siguiendo el modelo de la clase `DoublingRatio` de Sedgewick y Wayne [1]. La semilla se define como el exponente t multiplicado por 1000, lo que garantiza reproducibilidad entre ejecuciones. Los valores de la colección se generan en el rango $[0, 2^{t+2}]$ y el valor k se elige en $[1, \text{máx}(1,\ 2^{t-2})]$. Para cada instancia se genera una lista base y se copia tres veces, una por cada método a comparar, ya que los métodos 1 y 2 utilizan la función `.sort`, esto cambiaria el orden de la lista original.

El tiempo se mide con la clase `StopwatchCPU`, que registra el inicio y el fin de cada método y calcula la diferencia. Los resultados se exportan mediante la clase `Out`, que genera un archivo CSV por cada tamaño n con los tiempos de los tres métodos. Las clases `StdRandom` y `Result` se encargan de generar números aleatorios en un rango dado e implementar los tres métodos de búsqueda.

1.2. Desafíos enfrentados

El primer desafío fue la medición del tiempo. La clase `StopwatchCPU` original utiliza `getCurrentThreadCpuTime` que no entrega resultados confiables en entornos Windows. Esta función tiene una resolución de 15,625 ms [2], lo que hace que tiempos menores a ese valor se registren como 0,0 ms. Tras evaluar alternativas [3], se reemplazó por `System.nanoTime()`, que mide en nanosegundos y tiene alta resolución en todos los sistemas operativos, resultando más estable y preciso.

El segundo desafío fue generar valores sin repetición de forma eficiente. Se optó por `HashSet`, que maneja esto de forma automática y mantiene el código simple y legible.

Por último, la clase `StdRandom` incluía un método `main` propio que generaba errores de compilación al ejecutar el programa. La solución fue eliminarlo, dado que no era necesario para el funcionamiento del experimento. A continuación se muestra el error obtenido al compilar con las clases originales de Sedgewick:

Muestra del error:

```
PS C:\project\src> javac *.java
StdRandom.java:673: error: cannot find symbol
        StdOut.println(...);
        ^
  symbol:   variable StdOut
12 errors
```

1.3. Análisis empírico

Los promedios muestran que para tamaños pequeños (hasta $n = 128$) los tiempos son inconsistentes, lo que se atribuye al calentamiento de la JVM (*JVM warm-up*). A partir de $n = 256$ el comportamiento se estabiliza y las diferencias entre los tres métodos se vuelven apreciables, como se observa en la Figura 2.

Los histogramas indican que la mayoría de los tiempos se concentran en valores pequeños, lo que refleja consistencia en las mediciones, como se puede observar en la Figura 1. Las observaciones atípicas pueden explicarse por interferencias del sistema operativo o por el efecto de calentamiento de la JVM en instancias de tamaño reducido.

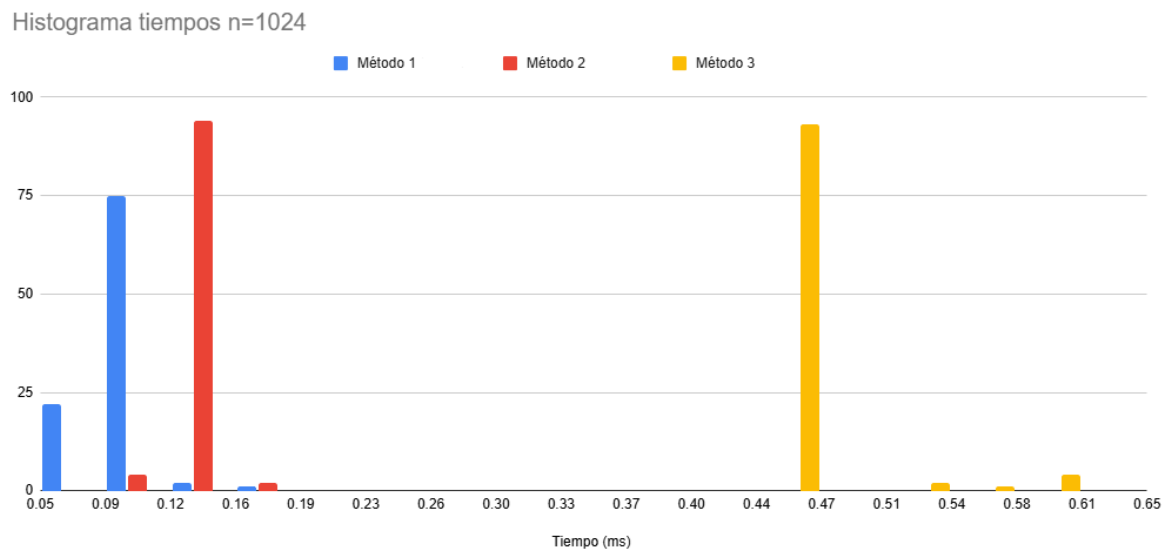

Figura 1: Histograma de tiempos de ejecución para $n = 1024$.

El gráfico log-log presenta las curvas de los tres métodos. Los métodos 1 y 2 crecen de forma similar y moderada, mientras que el método 3 exhibe una pendiente notoriamente mayor que se separa del resto a partir de $n = 256$, evidenciando un crecimiento más acelerado.

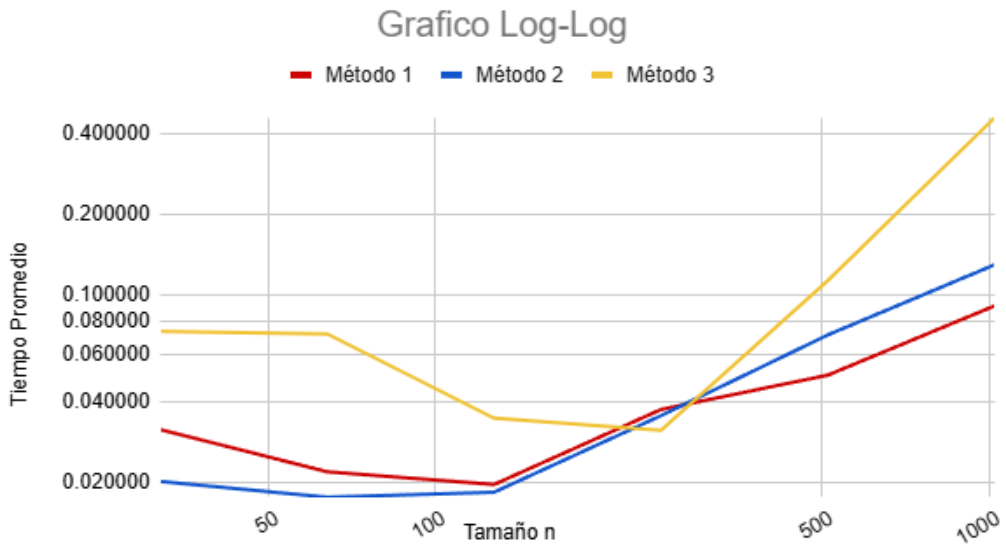

Figura 2: Gráfico log-log: tiempo promedio de ejecución vs tamaño n.

Al duplicar el tamaño de la instancia, el método 3 tarda aproximadamente cuatro veces más, lo que corresponde a un $\log_2$ del ratio cercano a 2, confirmando su complejidad $O(n^2)$. Los métodos 1 y 2, en cambio, tardan aproximadamente el doble al duplicar n, con un $\log_2$ del ratio cercano a 1, confirmando su complejidad $O(n \log n)$. Estos resultados son consistentes con el análisis teórico realizado previamente.

2. Etapa 2: Resultados en excel

El archivo `analisis.xlsx` está organizado y distribuido en siete pestañas: una por cada tamaño de entrada **(n = 32, 64, 128, 256, 512, 1024)** y una pestaña final llamada **Resumen**.

Cada pestaña de tamaño contiene los 100 tiempos medidos por método, junto con sus estadísticas descriptivas (mínimo, máximo, promedio, desviación estándar y coeficiente de variación) e histogramas de distribución.

Los histogramas de cada pestaña muestran que la mayoría de los tiempos se concentran en valores cercanos al promedio, lo que indica consistencia en las mediciones.

Los valores atípicos, presentes principalmente en tamaños pequeños, corresponden a las primeras ejecuciones afectadas por el *warm-up* de la JVM y por interferencias del sistema operativo. A partir de $n = 256$, las distribuciones se vuelven más estrechas y estables en los tres métodos.

La pestaña **Resumen** consolida los promedios de cada método por tamaño de entrada, presentados en la siguiente tabla:

n	Método 1 (ms)	Método 2 (ms)	Método 3 (ms)
32	0.0314	0.0201	0.0730
64	0.0218	0.0176	0.0714
128	0.0196	0.0183	0.0346
256	0.0373	0.0354	0.0311
512	0.0501	0.0707	0.1129
1024	0.0908	0.1293	0.4566

Tabla 1: Promedios de tiempo de ejecución por método y tamaño de entrada.

El análisis de $\log_2(ratio)$ entre tamaños consecutivos muestra que los Métodos 1 y 2 convergen progresivamente hacia 1 a medida que n crece, lo que es consistente con una complejidad $O(n \log n)$. En el intervalo más representativo ($n = 512 \rightarrow 1024$), el Método 1 obtiene un ratio de 1,81 con $\log_2 = 0{,}86$, y el Método 2 un ratio de 1,83 con $\log_2 = 0{,}87$, los dos aproximándose al valor teórico esperado de 1.

El Método 3, en cambio, alcanza valores de 1,858 y 2,016 en los intervalos $n = 256 \rightarrow 512$ y $n = 512 \rightarrow 1024$ respectivamente, convergiendo con claridad hacia 2 y confirmando su comportamiento $O(n^2)$. En ambos casos, las variaciones observadas en tamaños pequeños se explican por el calentamiento de la JVM, efecto que se disipa conforme n aumenta.

3. Etapa 3: Análisis teórico

Este análisis evalúa la eficiencia computacional de tres implementaciones en Java diseñadas para contar pares con una diferencia absoluta igual a k. Utilizando la notación Big-O [4], se estudiará el crecimiento del tiempo de ejecución en el peor caso en función del tamaño de la entrada n.

El estudio se centrará exclusivamente en el comportamiento asintótico de cada método, considerando únicamente las estructuras de control que determinan su crecimiento —como bucles y ordenamientos— y omitiendo operaciones auxiliares como inicializaciones de variables o retornos, cuyo impacto no varía con n. El objetivo es derivar la complejidad total de cada implementación y determinar teóricamente cuál resulta más eficiente.

3.1. Método 1: Función cuentaPares

Idea general: El método ordena la lista de valores y luego utiliza dos punteros (izq y der) que avanzan simultáneamente sobre ella. En cada paso, compara la diferencia entre los elementos apuntados con k: si es igual, cuenta el par; si es menor, avanza el puntero izquierdo; si es mayor, avanza el derecho. De este modo, recorre la lista una sola vez tras el ordenamiento.

Las líneas clave que determinan su complejidad son el ordenamiento inicial y el recorrido con punteros. En particular, el comportamiento lineal del bucle se puede verificar directamente en su cuerpo: en cada rama del condicional, al menos uno de los punteros avanza, por lo que nunca retroceden:

```java
pValores.sort(null);  // O(n log n)

while (der < pValores.size()) {  // O(n)
    if (pValores.get(izq) + pK > pValores.get(der)) {
        der += 1;
    }
    else if (pValores.get(izq) + pK == pValores.get(der)) {
        res += 1; izq += 1; der += 1;
    }
    else if (pValores.get(izq) + pK < pValores.get(der)) {
        izq += 1;
    }
}
```

$$Total = O(nlogn) + O(n) = O(nlogn). \tag{1}$$

Justificación:
El ordenamiento domina la complejidad total. El bucle while es lineal porque los punteros izq y der solo avanzan hacia adelante, de modo que entre ambos recorren la lista como máximo n veces. El término dominante cuando n $\rightarrow \infty$ es O(n log n), ya que crece más rápido que O(n); por ello, el término lineal se descarta junto con cualquier constante multiplicativa, y la complejidad total queda definida como O(n log n).

3.2. Método 2: Función cuentaPares2

Idea general: Al igual que el Método 1, ordena previamente la lista. Luego, por cada elemento de la lista, calcula cuánto debería valer su par —sumándole k— y verifica si ese valor existe en la lista mediante búsqueda binaria. Si la búsqueda es exitosa, contabiliza el par.

A diferencia del Método 1, que localiza los pares en un único recorrido con dos punteros, este método lanza una búsqueda binaria independiente por cada elemento, lo que explica su mayor constante oculta pese a tener la misma complejidad asintótica.

Las líneas clave que determinan su complejidad son el ordenamiento inicial, el bucle externo y la búsqueda binaria ejecutada en cada iteración:

```
pValores.sort(null);  // O(n log n)

for (int i = 0; i < pValores.size(); i++) {  // O(n)
    if (Collections.binarySearch(pValores,
            pValores.get(i) + pK) >= 0)  // O(log n)
        res += 1;
}
```

$$Total = O(n \log n) + O(n \log n) = O(n \log n). \tag{2}$$

Justificación:
La operación que más se repite es la búsqueda binaria: se ejecuta una vez por cada uno de los n elementos del bucle, acumulando un costo de O(n log n) para esa fase.

El término dominante cuando n $\to \infty$ es O(n log n), tanto del ordenamiento como del bucle con búsqueda binaria, por lo que los términos de menor orden y las constantes se omiten.

Aunque asintóticamente equivale al Método 1, en la práctica presenta una constante oculta mayor: realiza n búsquedas binarias independientes —cada una recorre el espacio de búsqueda desde cero—, mientras que el Método 1 hace un único recorrido lineal tras ordenar.

3.3. Método 3: Función cuentaPares3

Idea general: A diferencia de los métodos anteriores, este no requiere ordenamiento previo. Recorre la lista con dos bucles anidados: el externo fija un elemento i y el interno compara ese elemento contra todos los que le siguen; Por cada par (i, j), calcula su diferencia absoluta y verifica si es igual a k.

Es el enfoque más directo e intuitivo de los tres, pero también el más costoso: al no apoyarse en ninguna estructura auxiliar como ordenamiento o búsqueda binaria, debe examinar exhaustivamente todas las combinaciones posibles.

Las líneas clave que determinan su complejidad son los dos bucles anidados y la comparación ejecutada en cada par:

```java
for (int i = 0; i < pValores.size(); i++) {  // O(n)
    for (int j = i + 1; j < pValores.size(); j++) {  // O(n)
        if (Math.abs(pValores.get(i) - pValores.get(j)) == pK)
            res += 1;
    }
}
```

$$Total = O(n^2) \tag{3}$$

Justificación:
La operación que más se repite es la comparación `Math.abs(...) == pK`, ejecutada una vez por cada par (i, j).

El bucle externo recorre los n elementos de la lista y, por cada uno de ellos, el bucle interno recorre todos los elementos restantes a su derecha, generando en total $n(n-1)/2$ comparaciones.

Esta es la suma de una progresión aritmética cuyo término dominante cuando $n \to \infty$ es n^2; al eliminar la constante $1/2$ y el término de menor orden n, la complejidad queda definida como $\mathcal{O}(n^2)$.

Si bien este método evita el costo del ordenamiento previo, dicha ventaja es insignificante frente al crecimiento cuadrático impuesto por el doble bucle, convirtiéndolo en la implementación menos eficiente de las tres para entradas de gran tamaño.

4. Etapa 4: Comparaciones

La siguiente tabla resume, para cada método, la complejidad teórica esperada y el valor de $\log_2(ratio)$ observado en el intervalo más representativo ($n = 512 \rightarrow 1024$), donde el comportamiento asintótico ya domina sobre los efectos del calentamiento de la JVM.

Método	**Complejidad teórica**	$\log_2$ **esperado**	$\log_2$ **observado**	**¿Coincide?**
Método 1	$O(n \log n)$	≈ 1	0,859	Sí
Método 2	$O(n \log n)$	≈ 1	0,860	Sí
Método 3	$O(n^2)$	≈ 2	2,016	Sí

Tabla 2: Comparación entre complejidad teórica y comportamiento experimental en el intervalo $n = 512 \rightarrow 1024$.

Discusión:
Los tres métodos confirman experimentalmente su complejidad teórica: los Métodos 1 y 2 presentan un $\log_2(ratio)$ cercano a 1, consistente con $O(n \log n)$, mientras que el Método 3 alcanza 2,016, convergiendo con precisión al valor esperado de 2 para $O(n^2)$.

Las diferencias observadas en intervalos de n pequeño se explican por las siguientes causas:

- **Calentamiento de la JVM:** Para tamaños pequeños, la JVM aún no ha compilado el código al vuelo, lo que introduce tiempos artificialmente altos o inconsistentes que distorsionan los ratios.
- **Constantes ocultas:** La notación *Big-O* omite las constantes multiplicativas. Para n pequeño, estas constantes dominan sobre el término asintótico, haciendo que los tiempos no reflejen aún el orden de crecimiento real.
- **Efectos del hardware y del sistema operativo:** Interrupciones del sistema, uso de caché y planificación de procesos pueden introducir variaciones en las mediciones, especialmente cuando los tiempos absolutos son muy pequeños (del tipo de 0,02 ms).
- **Tamaños no suficientemente grandes:** El comportamiento asintótico solo se manifiesta con claridad cuando n es lo suficientemente grande como para que el término dominante supere al resto. En este experimento, dicho umbral se alcanza aproximadamente a partir de $n = 256$.

Resolución: Los resultados prácticos son consistentes con el análisis teórico en los tres casos. Las fluctuaciones observadas en valores pequeños de n no contradicen la teoría, sino que reflejan la influencia de factores externos como el calentamiento de la JVM, el garbage collector y los efectos de caché, los cuales pierden relevancia relativa a medida que n crece y el comportamiento asintótico domina.

5. Conclusiones

En primer lugar, se aprendió a utilizar la **notación Big-O** como herramienta formal para describir el crecimiento del tiempo de ejecución de un algoritmo. Se comprendió que Big-O no mide el tiempo exacto, sino el comportamiento asintótico cuando n crece, ignorando constantes y términos de menor orden. Esto permite comparar algoritmos de forma independiente del hardware o del lenguaje utilizado.

En segundo lugar, se aprendió a analizar la complejidad de métodos concretos identificando las estructuras de control dominantes: un bucle simple es $O(n)$, dos bucles anidados son $O(n^2)$, y un ordenamiento es $O(n \log n)$. Se comprendió además que cuando se combinan varias operaciones, el término de mayor orden domina y los demás se descartan.

En tercer lugar, se adquirió experiencia práctica en el uso de clases y funciones de Java no manejadas previamente. Entre ellas: `System.nanoTime()` para medición de tiempos de alta precisión y `HashSet` para garantizar valores sin repetición. Además, se trabajó con clases externas de la librería de Sedgewick y Wayne, integrándolas al proyecto y adaptándolas cuando fue necesario.

De los tres métodos analizados, el **Método 3** es el que crece más rápido. Su complejidad cuadrática $O(n^2)$ implica que, al duplicar el tamaño de la entrada, el tiempo de ejecución se cuadruplica. Esto se debe a que su estrategia recorre todos los pares posibles de elementos mediante bucles anidados, generando un número de operaciones proporcional a n^2. Para $n = 512$ tardó en promedio 0,113 ms, y al pasar a $n = 1024$ ese tiempo saltó a 0,457 ms, multiplicándose por un factor de 4, con un $\log_2 = 2$.

Finalmente, comparar teoría y práctica es esencial para validar el análisis. Un algoritmo puede tener una complejidad definida en papel, pero si los tamaños de entrada son pequeños, los costos fijos del sistema pueden ocultar su naturaleza real. En este experimento eso se observó con claridad: para $n = 32$ a $n = 256$ los ratios fueron inestables y no seguían los patrones esperados. Este comportamiento solo se volvió consistente a partir de $n = 512$, donde el término dominante superó la influencia de factores externos como el calentamiento de la JVM.

Referencias

[1] R. Sedgewick y K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011. Sección 1.4: “Analysis of Algorithms”.

[2] Microsoft Corporation, *Acquiring high-resolution time stamps*. Disponible en: https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps. Accedido: marzo 2026.

[3] Oracle Corporation, *Java Platform, Standard Edition Documentation*. Disponible en: https://docs.oracle.com/en/java/. Accedido: marzo 2026.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Capítulo 3: “Growth of Functions”.